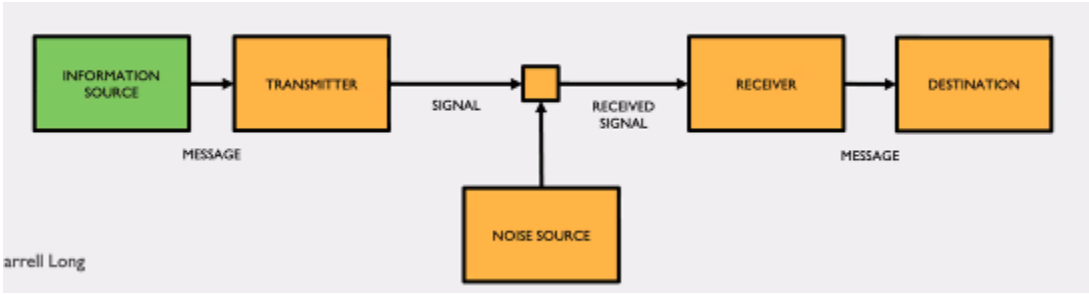
Darren Lee
CSE13S

Week 8 Notes

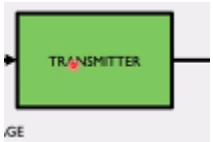
5/17/21
Data Compression
Communication

- Claude Shannon, the “father of modern information theory,” breaks down communication into 5 parts
- 1. Information source
- 2. Transmitter
- 3. Channel
- 4. Receiver
- 5. Destination

Information source

- Produces a message, or a sequence of messages, to be communicated to the receiving terminal
- A message takes on various forms
    - A sequence of symbols like in a telegraph or teletype system
    - A single function of time likke radio
    - A function of time and other variables like black/white television
    - 

Transmitter

- Operates on the message to produce as ginela suitable for transmission over the channel
- This is where compression and/or encryption occurs
- 

Channel

- The medium through which a signal is transmitted
- Could be, but not limited to:
    - Radio frequency
    - Beams of light

- Coaxial cables
- Wires
- Fiber optics

Receiver

- Performs the inverse operation of that done by the transmitter
- Decompression

Entropy

- Defined by Shannon as a measure of uncertainty of the occurrence of an event.
- Assume a set of possible events with probabilities $p = (p_1, p_2, \cdots, p_n)$ such that $\sum_{i=1}^{n} p_i = 1$.
  - That is, the probability of event e_i occurring is p_i for all $1 \leq i \leq n$.
- If such a measure exists, call it $H(p_1, p_2, \cdots, p_n)$, it can be required to have the following properties:
  1. H is continuous in p
  2. If all p_i equal, $p_i = \frac{1}{n}$, thus H is a monotonically increasing function of n.
     - There is some amount of uncertainty that occurs given equally likely events.
     - The more events, the more uncertainty.
  3. If a choice is broken down into two successive choices, the original H is the weighted sum of the individual values of H.
- Thus, $H = -\sum_{i=1}^{n} p_i \log_2(p_i)$
  - We call this entropy.

What does it all mean?

- Assume the following three messages:
  1. $AAAA$
  2. $AABC$
  3. $ABCD$

- Which message, if we pick a random symbol and guess what it is, has:

Better understanding entropy

- Consider message (1) which is AAAA
  - p(A)=1
- Using the formula for entropy we see that
- $$\begin{aligned} H &= -\sum_{i=1}^{n} p_i \log_2(p_i) \\ &= -1 \log_2(1) \\ &= 0 \end{aligned}$$
- Which means the entropy for this message is 0

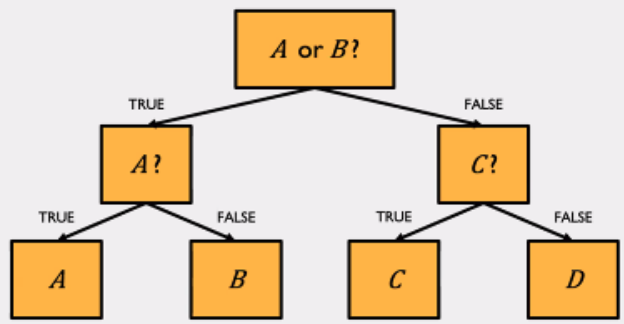
- Consider message (2), which is $AABC$.
  - $p(A) = \frac{1}{2}, p(B) = \frac{1}{4}, p(C) = \frac{1}{4}$
- Using the formula for entropy we see that

$$\begin{aligned} H &= -\sum_{i=1}^{n} p_i \log_2(p_i) \\ &= -\frac{1}{2}\log_2\left(\frac{1}{2}\right) - \frac{1}{4}\log_2\left(\frac{1}{4}\right) - \frac{1}{4}\log_2\left(\frac{1}{4}\right) \\ &= \frac{3}{2} \end{aligned}$$

- Which means the entropy for this message is $\frac{3}{2}$.

Guessing a symbol

- We can think of entropy as the average number of questions needed to be asked to correctly guess what random symbol we picked from the message.
- Consider message (3), which is $ABCD$.
- The best way to guess a random symbol randomly selected out of this message is to simulate binary search.
  - Is the symbol A or B?
    - If yes, then is it A?
    - Else, it must be B.
  - Else, the symbol must be either C or D.
    - Is it C?
    - Else, it must be D.
- Thus the average number of questions asked is 2, which is exactly the entropy we calculated for this message!



Run-length coding

- Idea: split up a message into sequences of identical symbols
  - These sequences are referred to as runs
  - Runs are represented as a single instance of the repeated value followed by a repetition count

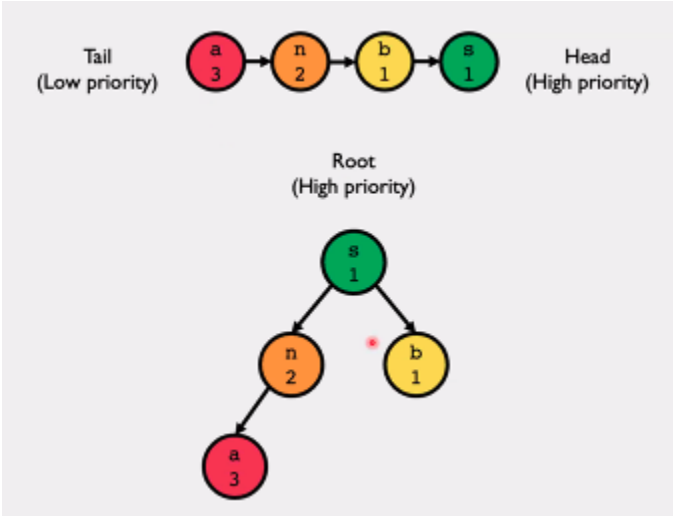
Huffman coding

- Developed by David A. Huffman
  - Distinguished member of the UCSC faculty
  - One of the founders of UCSC's COmputer Science department
- Idea: assign each symbol a unique bit-string code
  - Generate a histogram of unique symbols in data
  - The more frequently a symbol appears, the shorter its code
- So how does it work

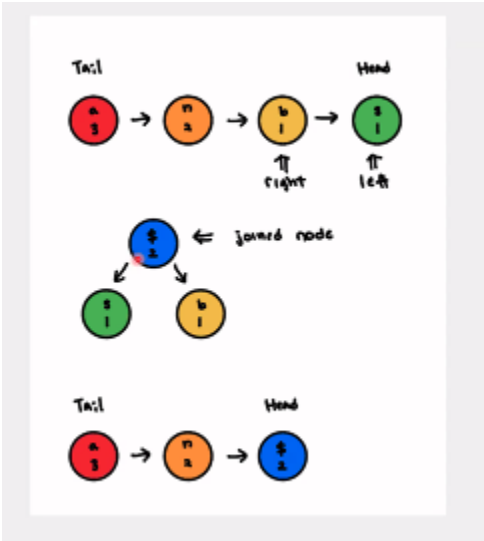
Constructing a histogram

- We'll compress the message "bananas"
- First, we create a histogram of the message's unique symbol and their frequencies

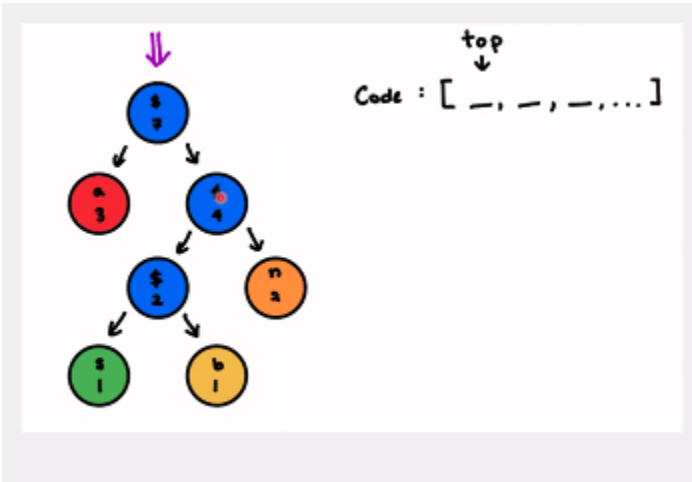
Priority queueing

- Each symbol with a non-zero frequency uis added to a priority queue as a node
- The lower the frequency, the higher the priority
- In a priority queue, the only guarantee is that the highest priority element is first
- 

Building a huffman tree

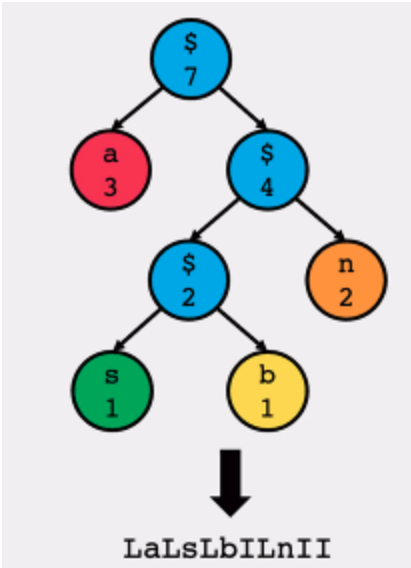
- We will create a huffman tree using the nodes in the priority queue
- While the queue has more than one node
- 1. Dequeue a node to the left child
- 2. Enqueue another node to the right hild
- Create a parent node for the left and right child nodes the frequency is the sum of the childrens' frequencies
- 4. Enqueue the parent node
- The last node in the queue is the root of the tree
- 

Building the code table

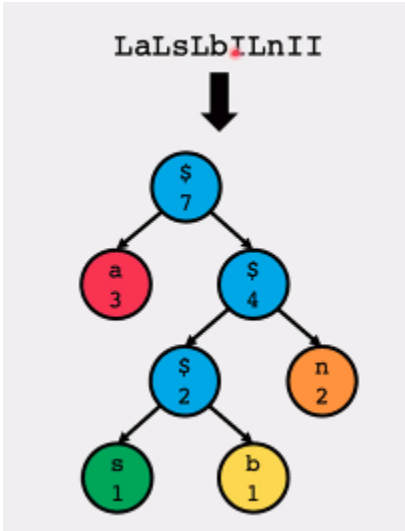
- We now populate a table of codes
    - Like the histogram there are 256 indices, and each index stores a binary code
- We will utilize a stack of bits to keep track of the code
- To build these codes, we perform a post-order traversal
    - If we walk down to the left child, we push a 0 to the bit stack
    - If we walk down to the right child, we push a 1 to the bit stack
    - If we reach a leaf node, the code for the node's symbol is the code in teh bit-stack
- A code can be, at maximum, 256 bits long
- 

Dumping the tree

- Some versions of Huffman coding emit the constructed tree
    - The tree is dumped to the coding through a post order traversal
- Performing a post order traversal of the tree
    - If a leaf node is reached, output an L followed by the node's symbol
    - If an interior node's children have been traversed, output an 'I' to indicate an interior node
- The number of symbols representing the tree dump is (3 x leaves)-1

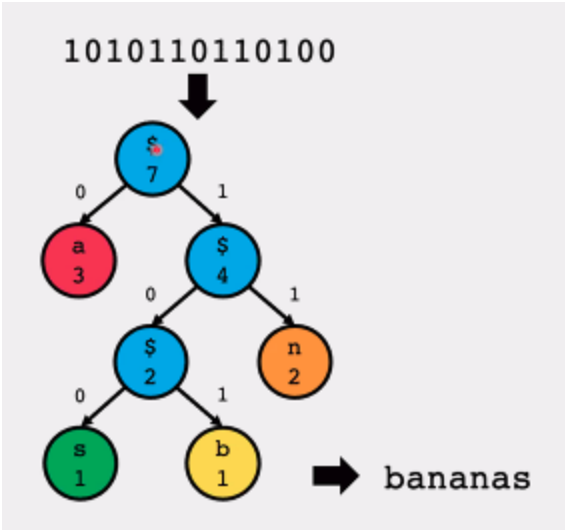


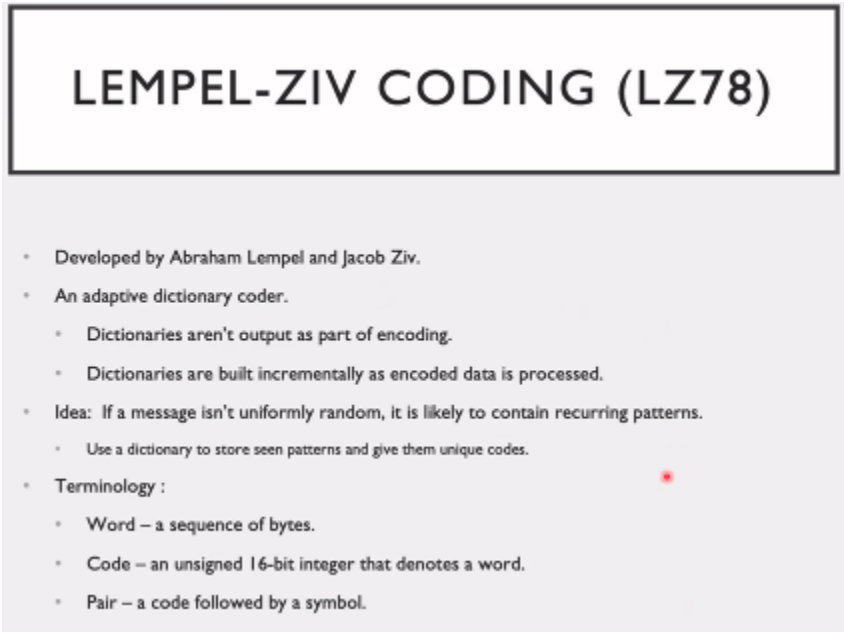
Reconstructing the tree

- To decompress the compressed message, we first reconstruct the tree from its dump
    - Recall that the dump was performed through post-order traversal
    - 

Decoding bit-by-bit

- To decode the binary codes, we walk the tree
- Read in each bit from the input
    - If the bit is a 0, walk down to the left child
    - If the bti is a 1, walk down to the tight child
    - If a leaf node is reached output its symbol and restart from the root of the tree

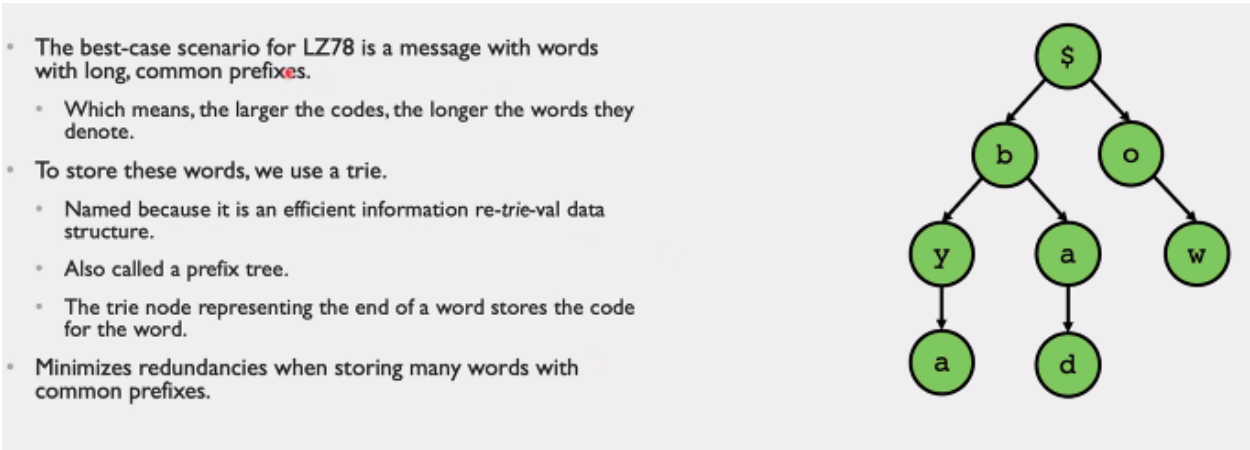
- 

- 

LEMPEL-ZIV CODING (LZ78)

- Developed by Abraham Lempel and Jacob Ziv.
- An adaptive dictionary coder.
  - Dictionaries aren't output as part of encoding.
  - Dictionaries are built incrementally as encoded data is processed.
- Idea: If a message isn't uniformly random, it is likely to contain recurring patterns.
  - Use a dictionary to store seen patterns and give them unique codes.
- Terminology :
  - Word – a sequence of bytes.
  - Code – an unsigned 16-bit integer that denotes a word.
  - Pair – a code followed by a symbol.

A dictionary of words



- The best-case scenario for LZ78 is a message with words with long, common prefixes.
  - Which means, the larger the codes, the longer the words they denote.
- To store these words, we use a trie.
  - Named because it is an efficient information re-*trie*-val data structure.
  - Also called a prefix tree.
  - The trie node representing the end of a word stores the code for the word.
- Minimizes redundancies when storing many words with common prefixes.

LZ78 ENCODING

- The message to encode is "ababababa".
- Initialize the trie to just the root node.
  - It will be given the code `EMPTY = 1` since the root denotes the empty word.
- We will need to keep track of the current node, which is colored: .
- Nodes that are added to the trie at any step are colored: .
- All other uninteresting nodes are colored: .

SUMMARY

- Two lossless compression algorithms:
  1. Huffman Coding
  2. LZ78
- Knowing that messages exhibit varying levels of entropy, it is good to pick your tools wisely.
  - Would it be good to use LZ78 for messages with high entropy?
  - Would it be better to use Huffman over LZ78 for message with low entropy?
- Compression algorithms have their limits.
  - They can't hope to compress messages consisting of uniform randomness.

5/19/21

Scripting

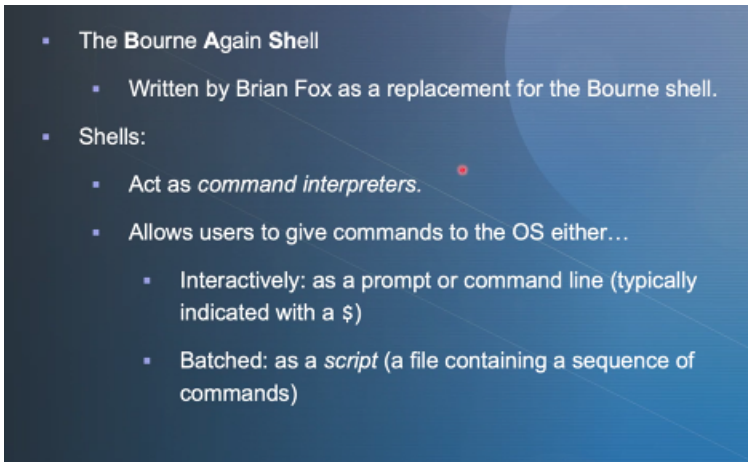
- Scripting lets the lazy be productive
- Operates on higher level objects
  - Files
  - Processes
- The *Software Tools Philosophy* lets us:
  - Connect the output of one program to the input of another program
  - This is program composition like (F o g o h)(x) from math

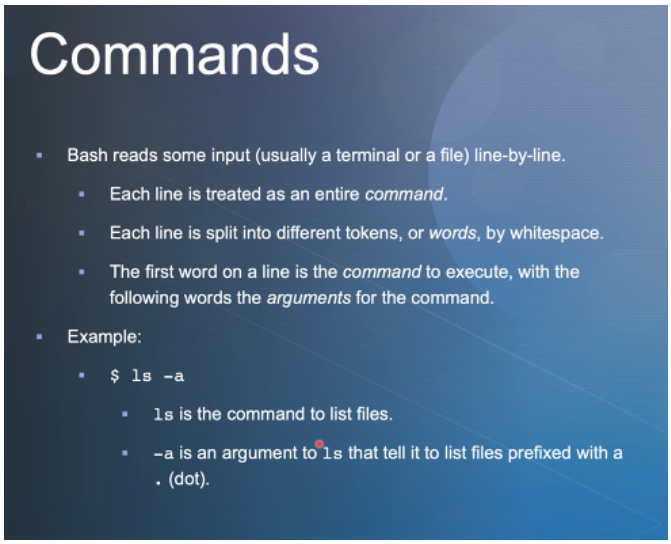
- Smart programmers reuse code, programs, components
- If i can use sort to sort a file, why should i rewrite it
- If i can use sed to make edits in a file, why should I write an editor
- If i can use grep to find all regular expressions in a file
- If I can use awk to parse a file

From sh to zsh

- Sh was the original UNIX shell
    - Called the Bourne shell, after Stephen Bourne
- There are many shells
    - Csh, tcsh, bash, zsh
    - You can write your own

BASH





Types of commands

- Aliases
    - Only used in interactive shells
    - Aliases are replace with what they are aliasing before a command is executed

        - Eg if ll is an alias for ls -l, then ll is replaced with ls -l before being executed interactively
    - Builtins
        - Bash-provided commands, such as printf, echo, and cd
    - Functions
        - A sequence of command that performs some task
        - Bash functions can accept arguments
    - Executables
        - Executable programs are specified and executed by file path
            - Or just by their respective names if the directory they're contained in are in the $PATH environment variable

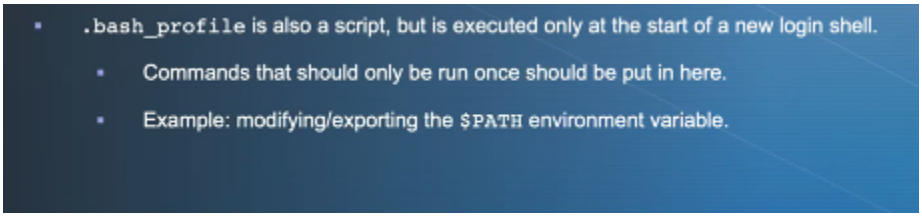
$PATH

- A colon-delimited list of directory names
    - These directories indicate where executable programs are located
- Directories typically included in $PATH
    - /bin - for commands required by system for repairs and booting
    - /usr/bin - primary directory of executables on the system
- The current directory . should not be included in your $PATH for security
    - Which is why executing your own programs typically requires a relative path
        - $ ./hello

Writing a script

- A script is a sequence of commands in a file (usually made executable)
- The first line of Bash script should be the interpreter directive
    - Also referred to as a shebang or a shebang
    - Indicates which interpreter to interpret the script with
    - #!/bin/bash - interpret script using /bin/bash

Basic dotfiles

- Dotfiles are files prefixed with a dot (.) that sit in your home directory
- There are 2 that directly relate with Bash (there are variants for different shells)
    - 1. .bashrc
    - 2. .bash_profile
- .bashrc is a script that is executed whenever Bash is started interactively
    - Commands that should be run every time a new interactive shell is started go here
    - Such as customized shell prompts, alisase, etc
    - 

Parameters and variables

- Used to store data
- A variable is a kind of parameter
    - The fact that a variable has a name is the distinguishing factor
- Variable names can only consist of letters. Digits, and underscores, and only start with either a letter or an underscore
- Assign data using the assignment operator =
    - Cannot use spaces around = in an assignment
- Variables can be strings, integers, Or arrays

Expansion

- Can expand parameters using the expansion operator $
- Variable expansion substitutes the variable with its value
-

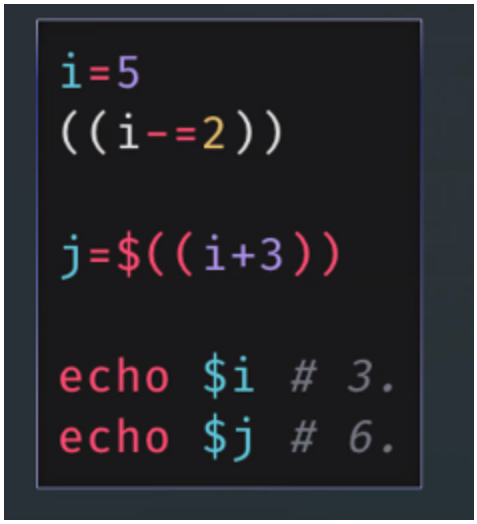
```
msg="This is a message."

# Echoes out "This is a message."
echo $msg
```

Arithmetic

- Arithmetic expressions are performed in ((...))
- The result of these expressions can be expanded with $((…))
- Some arithmetic operators (non exhaustive)
-

```
var++, var--
++var, --var
+, -, *, /, %
<, <=, =, !=, >=, >
&, ^, |, <<, >>
+=, -=, *=, /=, %=
```

- 

Special parameters

- Variables are named parameters
    - Special parameters are parameters that are not variables
- Examples:
    - $n - the n-th argument passed to the current script or function *these are positional parameters)
        - Use curly braces if n has more than one digit $(10)
    - $# - the number of positional parameters

Functions

- Sequences of commands
- Can have multiple functions within one script
- Allows for local variables
    - Prevents issue of overwriting variables from the function caller's scope

```
# Greets the first argument.
function greet() {
    echo "Hello $1!"
}

# Inelegant function to count files ending with ".txt".
function count_txts() {
    # Local variable to store count.
    local count=0

    # Loop through all files in directory.
    for f in *; do
        # Increase count if filename matches pattern.
        # More on globbing later.
        if [[ $f = *.txt ]]; then
            ((count+=1))
        fi
    done

    echo "Files ending with \".txt\": $count"
}

greet "CSE13S"
count_txts
```

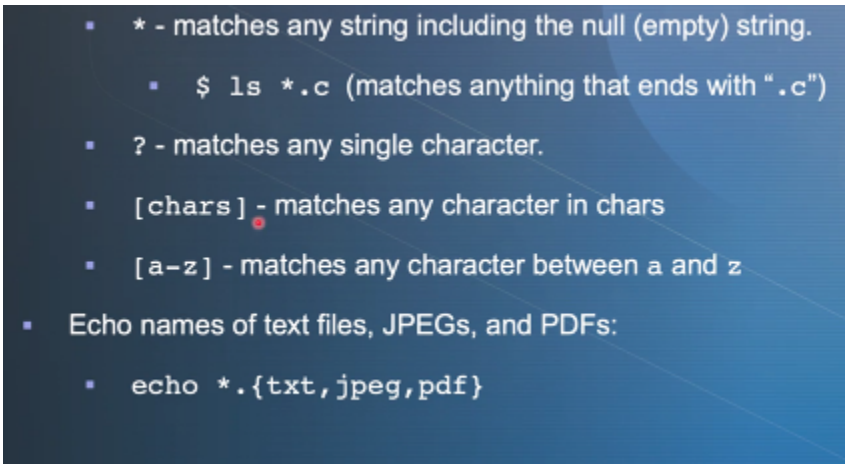
Arrays

- An array is a list of strings
- Most simply created using the arr=(..)
- Bash allows for arrays created using explicit indices for more flexibility
    - Arrays with gaps between indices and sparse arrays

```
# Array of fruits (known set size).
fruits=("apple" "banana" "cranberry")

# Array of treats using explicit indices.
treats=([0]="skittles" [1]="gummy bears" [4]="twix")
```

Globs and patterns

- Globs: patterns used to match strings
  - 

Testing and conditionals

- The logical operators that appear in c appear here in Bash as well
  - && - logical AND
  - || - locaical OR
  - ! - logical NOT
- Equality is tested using = (not == as is used in C)
- There are also if, elif, and else blocks
  - Conditional blocks must end with fi
- Tests are done within [[…]]
  - Can also use [..] but its colder and slacks features such as pattern matching

Testing (strings)

- Tests on strings
  - $s1 =$s2 (s1 match s2?)
  -
    - `$s1 != $s2` (s1 doesn't match s2?)
    - `$s1 < $s2` (s1 lexicographically less than s2?)
    - `$s1 > $s2` (s2 lexicographically greater than s2?)
    - `-z $s` (is s an empty string?)
    - `-n $s` (is s not an empty string?)
  -

```
s1="abc"
s2="abcd"

if [[ $s1 = $s2 ]]; then
  echo "s1 matches s2"
elif [[ $s1 < $s2 ]]; then
  echo "s1 < s2"
elif [[ $s1 > $s2 ]]; then
  echo "s1 > s2"
fi

s3=""
if [[ -z $s3 ]]; then
  echo "s3 is empty string"
else
  echo "s3 is not empty string"
fi
```

Testing (integers)

- Tests on integers

- $x -lt $y ($x < $y)
- $x -le $y ($x <= $y)
- $x -eq $y ($x = $y)
- $x -ne $y ($x != $y)
- $x -ge $y ($x >= $y)
- $x -gt $y ($x > $y)

- Can also test integers in arithmetic contexts.
  - Between ((…))

```
s1=7
s2=11

# In Bash test [[ ... ]].
if [[ $s1 -lt $s2 ]]; then
  echo "s1 < s2"
fi

# In arithmetic context.
if (( $s1 < $s2 )); then
  echo "s1 < s2"
fi
```

Loops

- Four different loop syntaxes in Bash
  - While
    - Loop while condition is true
  - Until
    - Loop until condition is true
  - For win word
    - Loop for each word of words, setting w as each word
  - For ((init; cond; incr))
    - Loop starting with the init expression, while the second expression is true, and incrementing

File redirection

- Each process on UNIX has access to the following file descriptors
  - Stdin - standard input

        - File descriptor 0
    - Stdout - standard outpu
        - File descriptor 1
    - Stderr - standard error
        - File descriptor 2
- These file descriptors can be redirected from or to files
-

```
# Redirects stdout of echo to hello.txt
echo "Hello!" > hello.txt

# Redirects hello.txt into stdin of cat.
cat < hello.txt

# Redirects stderr of program into err.txt.
# The 2 indicates the file descriptor to redirect.
./program 2> err.txt

# First, redirect program's stderr to its stdout.
# Then, redirect stdout to all.txt.
# This means that both stderr and stdout are redirected.
./program 2>&1 > all.txt

# Can also redirect stderr and stdout like so.
./program &> all.txt
```

Pipes

- Connects stdout of one process to stdin of another process
- Pipes are denoted with the pipe operator “ | “
- Pipes can be chained together, creating a pipeline
- Programs can be written such that they can be chained together to get interstging results
    - The whole philosophy behind the software tools that drive UNIX
- Example pipeline: list the IDs of the user’s processes (limit to 10)
    - Ps aux | grep $USER | awk ‘{ print $2 }’ | head

Why is called it called awk

- Awk is a program for simple parsing of text
    - It follows the *Software Tools Philosophy*
- It is named after
    - Albert Aho (a)
    - Peter Weinberger (w)
    - Brian Kernighan (k)

Awk

- Developed by Alfred Aho, Peter J. Weinberger, and Brian Kernighan
- Designed as a text processor that scans input files, splitting input lines into fields automatically
- Output from awk can be processed even further with other tools

A simple awk program

- A simple awk program is a sequence of pattern-action statements
    - Written as

        - Awk ‘pattern { action }’
    - Patterns are optional
- Suppose we have file of workers containing their names, the hosts they worked from the week, and their hourly wage
    - How would we process the file to only display the names of the workers who worked more than 10 hours during the week

More advanced patterns

- Pattern in awk can be combined (or even constructed using regular expressions)
    - logical operators
        - && - logical AND
        - || logical OR
        - ! logical NOT

Larger awk programs

- Awk is capable of much more than the simple text processing seen so far
    - Can write full-fledged programs in awk
- here’ s a simple program that counts the number of workers that worked more than 10 hours and less than 10 hours

Another awk example

- Another sample awk program to find and print out the longest line in some input.
- Makes use of:
    - **Arrays**
    - **Built-in functions**:
        - length(x): calculates length in characters of x.
    - **Built-in variables**:
        - NR: holds the number of lines currently read.

```
# longest.awk - Find longest line.

{
    # Save each input line into array.
    # $0 refers to the entire line.
    lines[NR] = $0

    # Update longest line and its line number.
    if (length($0) > len)
        len = length($0)
        linenr = NR
}

END {
    print "Longest line:", lines[linenr]
    print "Characters:", len
}
```

One last `awk` example

- Sorts input lines by increasing length.
- Makes use of
  - **Arrays**
  - **Functions**
    - Can accept parameters
    - Can return values
  - **Loops** (`while`/`for`)
    - Syntax of loops matches **C**-style loops.

```
# sortlines.awk - Sorts input lines by length.

# Inserts a line in the array lines.
# nlines refers to the number of lines in the array.
function insert(lines, nlines, line) {
    slot = nlines
    while (slot != 0 && length(lines[slot - 1]) > length(line)) {
        lines[slot] = lines[slot - 1]
        slot -= 1
    }
    lines[slot] = line
}

{
    # NR - 1 since NR starts the first line as 1.
    # Passing NR - 1 (0) means the lines array is empty.
    insert(lines, NR - 1, $0)
}

END {
    for (i = 0; i < NR; i += 1) {
        print lines[i]
    }
}
```

Useful commands

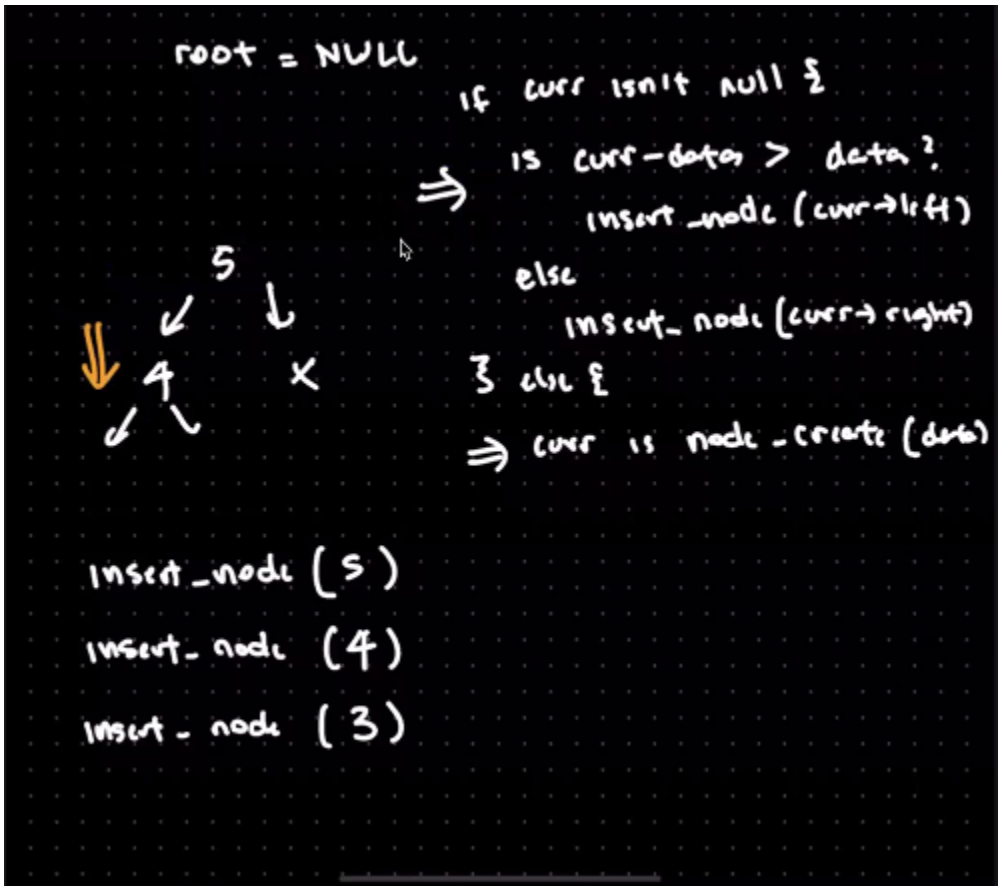
- `diff` – compare files line by line
- `tee` – copies `stdin` to `stdout`, copying the read contents to 0 or more files
- `sort` – sort or merge lines of files
- `uniq` – report or filter out duplicate lines in a file
- `wc` – report word, line, character, and/or byte count
- `mv` – move (or renames) a file
- `cp` – copy a file
- `rm` – remove a file

- Are DAGS
- (not all DAGS are trees)

Node
NULL

Binary trees have at most 2 children each

Leaf: a node w/o children
Non-leaf: internal/interior node



With binary search tree, search become O(logn) in a good case instead of O(n)

```
preorder_traversal (Node *n) {
    if (n isn't NULL) {
    ⇒ print out n's data
    → preorder_traversal (n→left)
    → preorder_traversal (n→right)
    }

inorder_traversal (Node *n) {
    if (n isn't NULL) {
        inorder_traversal (n→left);
        print out n's data
        inorder_traversal (n→right);
    }
```

```
postorder (Node *n) {
    if (n isn't NULL) {
        postorder (n→left)
        postorder (n→right)
        print out n's data
    }
```

case: removing a leaf node
Just set the leaf to null

Case: has 1 child node
Get rid of the child and replace it with its child
Inorder traversal goes in a DFS order and goes to till the node has no child